

Introdução a Algoritmos

Aula 13b – Ponteiros Avançados

Raoni F. S. Teixeira · 2s/2026

Objetivos desta aula

Ao final desta aula, você deverá ser capaz de:

1. Declarar e usar ponteiros para ponteiro () para permitir que uma função modifique um ponteiro do chamador;
2. Representar um vetor de strings como char e percorrer seus elementos com um laço sobre índices de ponteiro;
3. Interpretar a assinatura `int main(int argc, char *argv[])` e acessar os argumentos de linha de comando no programa;
4. Ler qualquer declaração de ponteiro em C aplicando a regra da espiral e identificando o tipo resultante.

1 O problema que motiva o ponteiro para ponteiro

Na Aula 13a escrevemos uma função que aloca memória e retorna o ponteiro:

```
float *le_vetor(int n) {  
    float *v = malloc(n * sizeof(float));  
    /* ... preenche v ... */  
    return v;  
}
```

Isso funciona bem. Mas às vezes queremos que a função **preencha** um ponteiro que o chamador já tem — modificar a variável ponteiro do chamador, não apenas o conteúdo que ela aponta. Considere:

```
void aloca(float *v, int n) {  
    v = malloc(n * sizeof(float));    /* modifica a copia local de v */  
}  
  
int main() {  
    float *vetor = NULL;  
    aloca(vetor, 10);  
    /* vetor ainda e NULL aqui – malloc nunca chegou ao chamador */  
}
```

O problema é o mesmo da função troca da Aula 7: `v` é uma **cópia** do ponteiro `vetor`. Modificar `v` (fazer `v = malloc(...)`) não afeta `vetor` em `main`.

Para modificar uma variável do chamador, precisamos passar seu endereço. Se a variável é um `float` *, seu endereço é um `float **` — ponteiro para ponteiro.

2 Ponteiro para ponteiro

Definição — Ponteiro para ponteiro

Um ponteiro para ponteiro (**) armazena o endereço de um ponteiro. Desreferenciar uma vez (*p) acessa o ponteiro armazenado; desreferenciar duas vezes (**p) acessa o valor final apontado.

```
int    x    = 42;
int    *p    = &x;    /* p aponta para x          */
int    **pp   = &p;    /* pp aponta para p          */

printf("%d\n", x);    /* 42 – valor direto          */
printf("%d\n", *p);    /* 42 – via p              */
printf("%d\n", **pp); /* 42 – via pp -> p -> x    */

**pp = 99;    /* modifica x através de pp */
printf("%d\n", x);    /* 99                       */
```

Visualizando na memória:

Variável	x	p	pp
Endereço	1000	2000	3000
Valor	42	1000	2000

pp contém 2000 (endereço de p); p contém 1000 (endereço de x); x contém 42. **pp percorre a cadeia toda.

2.1 Resolvendo o problema da alocação

Com **, a função pode modificar o ponteiro do chamador:

```
void aloca(float **v, int n) {
    *v = malloc(n * sizeof(float)); /* modifica o ponteiro original */
}

int main() {
    float *vetor = NULL;
    aloca(&vetor, 10);    /* passa o endereço do ponteiro */
    if (vetor != NULL) {
        vetor[0] = 3.14;
        free(vetor);
    }
    return 0;
}
```

*v = malloc(...) escreve no endereço apontado por v — que é a variável vetor em main. Após a chamada, vetor aponta para o bloco alocado.

3 Vetor de strings: char

Na Aula 10 representamos strings como `char[]` — vetores de `char`. Um **vetor de strings** é um vetor de ponteiros para `char`: cada elemento aponta para uma string diferente.

3.1 Com array de arrays (tamanho fixo)

```
char nomes[5][100]; /* 5 strings de ate 99 caracteres cada */
```

Simples, mas desperdiçador: reserva 100 bytes para cada string mesmo que ela tenha 3 caracteres.

3.2 Com vetor de ponteiros (char)

```
char *frutas[] = {"maca", "banana", "laranja", "uva"};
```

Aqui `frutas` é um array de 4 ponteiros, cada um apontando para um literal de string no segmento de código. Compacto — cada ponteiro ocupa 8 bytes, e os literais ocupam exatamente o necessário.

Índice	Ponteiro (<code>frutas[i]</code>)	String apontada
[0]	endereço A	"maca\0"
[1]	endereço B	"banana\0"
[2]	endereço C	"laranja\0"
[3]	endereço D	"uva\0"

Percorrer o vetor de strings é um laço sobre ponteiros:

```
int n = 4;
for (int i = 0; i < n; i++)
    printf("%s\n", frutas[i]);
```

E percorrer os caracteres de uma string específica:

```
for (int j = 0; frutas[2][j] != '\0'; j++)
    printf("%c", frutas[2][j]); /* imprime "laranja" caractere a caractere */
```

`frutas[2][j]` é natural: `frutas[2]` é o ponteiro para “laranja”; `[j]` indexa o *j*-ésimo caractere.

3.3 Vetor de strings alocado dinamicamente

Para strings modificáveis e de tamanho variável, alocamos cada string no heap:

```
#include <stdlib.h>
#include <string.h>

char **aloca_strings(int n, int max_len) {
    char **v = malloc(n * sizeof(char *)); /* vetor de n ponteiros */
    if (v == NULL) return NULL;
    for (int i = 0; i < n; i++) {
        v[i] = malloc(max_len * sizeof(char));
        if (v[i] == NULL) {
```

```

        /* libera o que ja foi alocado antes de retornar NULL */
        for (int j = 0; j < i; j++) free(v[j]);
        free(v);
        return NULL;
    }
}
return v;
}

void libera_strings(char **v, int n) {
    for (int i = 0; i < n; i++) free(v[i]); /* libera cada string */
    free(v);                               /* libera o vetor de ponteiros */
}

```

A liberação é em dois níveis — primeiro cada string, depois o vetor de ponteiros — na ordem inversa da alocação.

4 argc e argv: argumentos da linha de comando

Desde a Aula 1 usamos `int main()`. A forma completa de `main` é:

```
int main(int argc, char *argv[]) { ... }
```

ou equivalentemente:

```
int main(int argc, char **argv) { ... }
```

- `argc` (**argument count**): número de argumentos passados ao programa, incluindo o próprio nome do executável. Sempre pelo menos 1.
- `argv` (**argument vector**): vetor de strings com os argumentos. `argv[0]` é o nome do programa; `argv[1]` é o primeiro argumento, etc.

```

#include <stdio.h>

int main(int argc, char *argv[]) {
    printf("Programa: %s\n", argv[0]);
    printf("Argumentos recebidos: %d\n", argc - 1);
    for (int i = 1; i < argc; i++)
        printf("  argv[%d] = %s\n", i, argv[i]);
    return 0;
}

```

Executando `./programa ola mundo 42`:

```

Programa: ./programa
Argumentos recebidos: 3
  argv[1] = ola
  argv[2] = mundo
  argv[3] = 42

```

`argv` é um `char **` — um ponteiro para o primeiro elemento de um vetor de ponteiros para `char`. É exatamente a estrutura que acabamos de estudar. `argv[argc]` é garantidamente `NULL` — um sentinela que permite percorrer os argumentos sem conhecer `argc`:

```

for (int i = 1; argv[i] != NULL; i++)
    printf("%s\n", argv[i]);

```

5 Exemplo integrador: ordenar strings recebidas como argumentos

O programa abaixo recebe palavras como argumentos de linha de comando e as imprime em ordem lexicográfica, usando `strcmp` para comparar.

```
#include <stdio.h>
#include <string.h>

/* Ordena vetor de strings por selecao (ordem lexicografica) */
void ordena(char *v[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (strcmp(v[j], v[min]) < 0)
                min = j;
        /* troca v[i] com v[min] — troca ponteiros, nao conteudo */
        char *temp = v[i];
        v[i] = v[min];
        v[min] = temp;
    }
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Uso: %s palavra1 palavra2 ...\n", argv[0]);
        return 1;
    }

    /* argv[1..argc-1] sao as palavras; passamos o subvetor */
    ordena(argv + 1, argc - 1);

    for (int i = 1; i < argc; i++)
        printf("%s\n", argv[i]);

    return 0;
}
```

Observe `argv + 1`: aritmética de ponteiros — `argv + 1` é o endereço do segundo elemento de `argv`, equivalente a `&argv[1]`. Passar `argv + 1` para `ordena` faz com que ela trate `argv[1]` como seu `v[0]`.

Também observe a troca: trocamos os **ponteiros** `v[i]` e `v[min]`, não os conteúdos das strings. Isso é eficiente — independente do comprimento das strings, a troca é sempre três atribuições de ponteiro.

6 Leitura de declarações complexas

À medida que os programas crescem, declarações de ponteiros tornam-se mais complexas. A regra para ler qualquer declaração em C é aplicar a **regra da espiral** da direita para a esquerda, priorizando `[]` e `()` antes de `*`:

Declaração	Leitura
<code>int *p</code>	“p é ponteiro para int”
<code>int *v[]</code>	“v é array de ponteiros para int”
<code>int **pp</code>	“pp é ponteiro para ponteiro para int”
<code>char *argv[]</code>	“argv é array de ponteiros para char (= array de strings)”
<code>int (*f)(int)</code>	“f é ponteiro para função que recebe int e retorna int”
<code>int *(*g)(int)</code>	“g é ponteiro para função que recebe int e retorna ponteiro para int”

As últimas duas linhas envolvem ponteiros para função — um tópico que aparece em callbacks e tabelas de despacho, abordado em disciplinas avançadas. O importante agora é saber ler a declaração corretamente.

7 Exercícios

1. Desenhe o diagrama de memória para o código abaixo, mostrando endereços fictícios para cada variável:

```
int x = 10, y = 20;
int *p = &x;
int **pp = &p;
*pp = &y;
**pp = 99;
printf("%d %d\n", x, y);
```

O que é impresso? Verifique compilando.

2. Escreva uma função `void troca_strings(char **a, char **b)` que troca dois ponteiros de string. Teste com:

```
char *s1 = "ola";
char *s2 = "mundo";
troca_strings(&s1, &s2);
printf("%s %s\n", s1, s2); /* deve imprimir: mundo ola */
```

3. Escreva um programa que receba um número inteiro `n` como argumento de linha de comando e imprima os primeiros `n` números primos. (Use `atoi(argv[1])` para converter o argumento de string para inteiro.)
4. Escreva uma função `char **le_palavras(int *n)` que: (a) leia `*n` do teclado; (b) aloque um vetor de `*n` ponteiros; (c) para cada posição, aloque exatamente `strlen + 1` bytes e leia uma palavra; (d) retorne o vetor. Escreva a função de liberação correspondente e use as duas em `main`.
5. (Desafio) Escreva `char **split(char *s, char sep, int *n)` que divide a string `s` pelo separador `sep`, retorna um vetor de substrings alocadas no heap e armazena o número de partes em `*n`. Exemplo: `split("um:dois:tres", ':', &n)` retorna `{"um", "dois", "tres"}` com `n = 3`.